实验 2. 基于动态规划发求解最长公共子序列等问题

班级：数科 2102　　姓名：陈　　学号：202101

实验内容

本实验要求基于算法设计与分析的一般过程（即待求解问题的描述、算法设计、算法 描述、算法正确性证明、算法分析、算法实现与测试），在针对最长公共子序列问题求解的 实践中理解动态规划（Dynamic Programming, DP）方法的思想、求解策略及步骤。**作为挑战：可以完成诸如 0-1 背包、矩阵连乘等问题。**

实验目的

- 理解动态规划方法的核心思想以及动态规划方法的求解过程；
- 从算法分析与设计的角度，对最长公共子序列问题的基于 DP 法求解有更进一步的理解。

环境要求

无特别要求。算法实现可以自由选择 C、C++、Java、Python 等。

最长公共子序列问题

实验步骤

1、 理解问题，给出问题的描述。
给定两个序列 X 与 Y，求二者的最长公共子序列 Z，若存在多个最长公共子序列，输出其一即可

2、 算法设计，包括算法策略与数据结构的选择；
分析可得：当$X_m = Y_n$时，则$Z_k = X_m = Y_n$，且Z_{k-1}为X_{m-1}1 与Y_{n-1}的最长公共子序列；
若$X_m \neq Y_n$且$Z_k \neq X_m$，则 Z 是X_{m-1}和 Y 的最长公共子序列；
若$X_m \neq Y_n$且$Z_k \neq X_m$，则 Z 是 X 和Y_{n-1}的最长公共子序列；
用 dp[i][j]记录X_i和Y_j的 LCS 的长度

3、 描述算法。希望采用源代码以外的形式，如**伪代码**、流程图等；

```
Solve(n, m)
  for i ← 0 to n
    do dp[i,0] ← 0
  for j ← 0 to m
    do dp[0,j] ← 0
  for i ← 1 to n
    do for j ← 1 to m
        do if x[i] = y[i]
            then dp[i,j] ← dp[i-1,j-1]+1
                 dir[i,j] ← 1
            else if dp[i-1,j] >= c[i,j-1]
                 then dp[i,j] ← dp[i-1,j]
                      dir[i,j] ← -1
                 else dp[i,j] ← dp[i,j-1]
                      dir[i,j] ← 2
```

4、 算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明；

在计算 X 和 Y 的 LCS 时，包含了计算X_{m-1}和 Y 与 X 和Y_{n-1}的 LCS，可得如下状态转移方程

$$dp[i,j] = \begin{cases} 0,\ i=0, j=0 \\ dp[i-1][j-1]+1, \qquad i,j>0; X_i = Y_j \\ max\{dp[i][j-1], dp[i-1][j]\},\ i,j>0; x_i \neq Y_j \end{cases}$$

最终可得问题的解

5、 算法复杂性分析，包括时间复杂性和空间复杂性；

由两层循环与 dp 数组大小可知：

时间复杂度：O(nm)

空间复杂度：O(nm)

实验结果

算法实现与测试，附上代码或以附件的形式提交，同时贴上算法运行结果截图；

```
#include <iostream>
#include <stack>
using namespace std;

const int N = 1e3+10;
string a = " ",b = " ",str;
int n,m;
int dp[N][N];
int dir[N][N]; // north:-1,northwest:1,west:2

void solve(int lenA, int lenB);
void print();
```

```
int main(){

	cin >> str;
	n = str.size();
	a += str;

	cin >> str;
	m = str.size();
	b += str;

	solve(n, m);
	print();
}

void solve(int lenA, int lenB){
	for (int i = 1; i <= lenA;i++) dp[i][0] = 0;
	for (int i = 1; i <= lenB;i++) dp[0][i] = 0;

	for (int i = 1; i <= lenA;i++){
		for (int j = 1; j <= lenB;j++){
			if(a[i] == b[j]){
				dp[i][j] = dp[i-1][j-1] + 1;
				dir[i][j] = 1;
			} else if (dp[i-1][j] >= dp[i][j-1]){
				dp[i][j] = dp[i-1][j];
				dir[i][j] = -1;
			} else {
				dp[i][j] = dp[i][j-1];
				dir[i][j] = 2;
			}
		}
	}
}

void print(){
	int x = n,y = m;
	stack<char> st;
	while(x > 0 && y > 0){
		if(dir[x][y] == 1){
			st.push(a[x]);
			x--;
			y--;
		} else if(dir[x][y] == -1){
			x--;
```

```
            } else {
                y--;
            }
        }
        while(!st.empty()) {
            cout << st.top();
            st.pop();
        }
        cout << endl;
    }
```

```
1a2b3c4d5e6f7g
123abcd456
1abc456
PS F:\Code>
```

0-1 背包

实验步骤

1、 理解问题，给出问题的描述。

n 个物品和 1 个背包。对物品 i，其价值为 vi，重量为 wi，背包容量为 W。如何选取物品装入背包，使背包中所装入的物品的总价值最大？其中，wi, W 都是正整数

2、 算法设计，包括算法策略与数据结构的选择；

设计数组 dp[i][j]为当背包剩余空间为 j，可选取物品为前 i 件物品时的物品最大总价值，故问题的解为 dp[n][W]，可知当背包装不下第 i 件物品时:

dp[i][j]=dp[i-1][j]

当装得下第 i 件物品时:

dp[i][j]=max{dp[i-1][j],dp[i-1][j-wi]+vi}

3、 描述算法。希望采用源代码以外的形式，如<u>伪代码</u>、流程图等；

```
Solve()
  for i 1 to n dp[i,0] = 0
  for j 1 to m dp[0,j] = 0

  for i 1 to n //物品数
    do for j 1 to m //背包容量
      do dp[i,j] = dp[i-1,j]
        if j >= w[i]
          then dp[i,j] = max{dp[i-1,j],dp[i-1,j-w[i]]+v[i]}
```

4、 算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明；

由状态转移方程可知及 dp[i][0] = dp[0][j] = 0 可知其正确性

$$dp[i][j] = \begin{cases} dp[i-1][j],\ j < w[i] \\ max\{dp[i-1][j], dp[i-1][j-w[i]] + v[i]\},\ j \geq w[i] \end{cases}$$

5、 算法复杂性分析，包括时间复杂性和空间复杂性；

由两层循环及 dp 数组大小可知：

时间复杂度：O(nm)

空间复杂度：O(nm)

实验结果

算法实现与测试，附上代码或以附件的形式提交，同时贴上算法运行结果截图**;**

```
#include <iostream>
using namespace std;

const int N = 1e3 + 10;

int dp[N][N];
int n, m;
int v[N], w[N];
bool chosen[N];

void solve();

int main() {
	cout << "请输入物品个数与背包大小：";
	cin >> n >> m;
	cout << "请依次输入每个物品的价值：";
	for (int i = 1; i <= n; i++)
		cin >> v[i];
	cout << "请依次输入每个物品的大小：";
	for (int i = 1; i <= n; i++)
		cin >> w[i];

	solve();

	return 0;
}

void solve() {
	for (int i = 1; i <= n; i++) {
		for (int j = 1; j <= m; j++) {
			if (j >= w[i]) {
				if (dp[i - 1][j] >= dp[i - 1][j - w[i]] + v[i]) {
					dp[i][j] = dp[i - 1][j];
				} else {
```

```
                dp[i][j] = dp[i - 1][j - w[i]] + v[i];
            }
        } else {
            dp[i][j] = dp[i - 1][j];
        }
    }
}
cout << "最多能拿价值之和为" << dp[n][m] << "的物品" << endl;

int j = m;
for (int i = n; i >= 1; i--) {
    if (dp[i][j] == dp[i - 1][j]) {
        chosen[i] = false;
    } else {
        chosen[i] = true;
        j -= w[i];
    }
}
cout << "拿取物品的编号为：";
for (int i = 1; i <= n; i++)
    if (chosen[i])
        cout << i << " ";
cout << endl;
}
```

```
请输入物品个数与背包大小：5 10
请依次输入每个物品的价值：6 3 5 4 6
请依次输入每个物品的大小：2 2 6 5 4
最多能拿价值之和为15的物品
拿取物品的编号为：1 2 5
```

实验总结

(1) 遇到哪些困难，如何解决的；

在列状态转移方程时较为困难，通过人工模拟小数据的动规的方法总结出状态转移方程

(2) 有哪些收获；

通过本次实验，我知道了在解决最优化等问题时，可以通过将子问题的解存储起来，并通过列出状态转移方程，最终得到问题的解。

(3) 本章所学到算法设计方法，能否举一反三，进行拓展；

动态规划问题最关键的是找出问题的状态转移方程，之后才能解决问题，除了线性 dp 外，还有状压 dp，概论 dp 等经典问题，这些思想在解决相关问题时十分好用

(4) 针对本章学习内容，还需要老师哪些方面的帮助.

暂无